Matematica Discreta
B-2026

PassShield

Evaluación de la fortaleza de políticas de autenticación mediante análisis combinatorio y el principio multiplicativo

Ferney Bermudez Toloza, Gabriel Diaz Argote, Paola Sanchez Rozo, Saray Silva Galindo.

Ingenieia en Ciencias de la Computacion e Inteligencia Artificial

Resumen

Las políticas de contraseñas de muchos sistemas de software se definen de forma empírica, sin una medida matemática que indique cuántas claves distintas permiten realmente ni cuánto resistirían un ataque de fuerza bruta. Esta carencia produce credenciales vulnerables y una falsa sensación de seguridad. Nuestro proyecto PassShield aborda este problema aplicando el análisis combinatorio, las permutaciones con repetición y el principio multiplicativo de la Matemática Discreta a la evaluación de políticas de autenticación. La solución se desarrolla en Python y HTML, y la metodología desarrollada calcula un espacio muestral universal de todas las combinaciones calculadas en función del tamaño del alfabeto permitido (N) y la longitud de la clave (L) proporcionando estimaciones del tiempo necesario para descifrar estas contraseñas bajo varios diseños de procesadores computacionales. Al final, el producto es un software interactivo con una demostración en tiempo real que calificará la fortaleza de una contraseña o política en cinco niveles, de Muy Débil a Muy Fuerte y las recomendaciones de optimización del espacio. El desarrollo sigue una metodología ágil adaptada a tres sprints y emplea únicamente la biblioteca estándar del lenguaje.

Palabras claves: Análisis combinatorio, principio multiplicativo, espacio muestral, permutaciones con repetición, seguridad de contraseñas, Python, HTML.

¿Cuánto tardaría un atacante en abrir tu contraseña, y qué dicen las matemáticas al respecto?

Introducción

La autenticación basada en contraseñas continúa siendo uno de los mecanismos de control de acceso más extendidos en aplicaciones web, sistemas empresariales y servicios en la nube. Su seguridad depende, entre otros factores, del tamaño del conjunto de claves que un atacante tendría que explorar para dar con la correcta. Las organizaciones suelen definir reglas de formación (longitud mínima, uso de mayúsculas, dígitos o símbolos) sin cuantificar el efecto que cada una tiene sobre ese conjunto. Cuando el atacante dispone de un procedimiento automatizado capaz de probar un gran número de combinaciones por segundo, la diferencia entre una política razonable y una deficiente puede abarcar varios órdenes de magnitud, y solo un análisis matemático permite medirla.

Nuestro proyecto se ubica en la Ingeniería de Inteligencia Artificial, con un componente transversal de Seguridad Informática o ciberseguridad, dentro del campo de las Ciencias de la Computación. Su fundamento matemático proviene de la Matemática Discreta, en particular de las técnicas de conteo como principios aditivo y multiplicativo, permutaciones con repetición, y se complementa con nociones elementales de teoría de la información, como la entropía.

Vale la pena resolver este problema porque la fortaleza de una política de contraseñas es una magnitud medible que, sin embargo, suele evaluarse de manera cualitativa. Disponer de una métrica exacta permite comparar políticas, justificar decisiones de diseño ante equipos técnicos y usuarios, y detectar reglas que aportan poco al espacio de búsqueda frente a otras que aportan mucho como por ejemplo, exigir un símbolo obligatorio frente a exigir mayor longitud. Además, el problema constituye un caso de estudio idóneo para mostrar cómo una función exponencial determina la viabilidad computacional de un ataque.

En este proyecto se formuló y diseñó PassShield, un programa en Python y HTML que evalúa políticas de contraseñas mediante análisis combinatorio. El trabajo comprendió:

(i) la definición del modelo matemático (alfabeto, espacio muestral, entropía y tiempo de ruptura)
(ii) la especificación de una arquitectura de tres componentes —clasificación de caracteres, motor combinatorio y generador de diagnóstico—
(iii) la formulación de los requerimientos funcionales y no funcionales
(iv) la planificación metodológica en tres sprints
(v) el diseño de un plan de validación basado en casos de prueba que contrastan contraseñas débiles con claves de alta entropía.

Objetivos

Objetivo general

Desarrollar un software en Python y HTML, fundamentado en el análisis combinatorio y el principio multiplicativo, que calcule el tamaño del espacio de búsqueda de una política de contraseñas, estime su tiempo de resistencia ante ataques de fuerza bruta y clasifique su fortaleza en cinco niveles, en un plazo de cinco semanas distribuidas en tres sprints.

Objetivos específicos

- Investigar y documentar las fórmulas del principio aditivo, el principio multiplicativo y las permutaciones con repetición, y aplícarlos al conteo del conjunto discreto de claves.
- Diseñar con Python y HTML una estructura lógica que separe los caracteres de una contraseña en conjuntos distintos (letras mayúsculas, letras minúsculas, dígitos y símbolos).
- Implementar un cálculo combinatorio para descubrir realmente cuántas combinaciones hay (N^L) y el tiempo estimado de descifrado según la debilidad del atacante.
- Evaluar la precisión de la herramienta en escenarios del mundo real probando contraseñas débiles frente a claves de alta entropía mediante una demostración interactiva.

Justificación y planteamiento del problema

En el desarrollo de aplicaciones web y empresariales, la autenticación mediante usuario y contraseña sigue siendo la puerta de entrada a datos personales, financieros y corporativos. Las plataformas establecen reglas de formación de contraseñas, pero rara vez disponen de una medida cuantitativa del nivel de protección que esas reglas ofrecen.

Ante políticas poco exigentes, los usuarios tienden a elegir contraseñas cortas o predecibles. Sin un análisis matemático claro, las organizaciones desconocen cuántas combinaciones reales protege su sistema y, por tanto, no pueden determinar si una regla es suficiente ni comparar alternativas de diseño. Esto deja abierta la puerta a ataques de fuerza bruta automatizados, capaces de probar millones de combinaciones por segundo.

Cuando una credencial es vulnerada, las consecuencias recaen sobre las personas, en situaciones de acceso no autorizado a cuentas personales, suplantación de identidad, pérdida o filtración de información y perjuicios económicos y de reputación. En el plano organizacional implica costos por incidentes de seguridad, pérdida de confianza de los usuarios y

posibles incumplimientos de normativas de protección de datos.

PassShield convierte una pregunta cualitativa en una respuesta cuantitativa concluyendo cuántas claves admite, cuántos bits de entropía aporta y cuánto tardaría un atacante en explorarlas todas bajo escenarios definidos. Esto permite diseñar, auditar y ajustar políticas antes de desplegarlas en producción.

Marco teorico

Alfabeto (N)

Es el conjunto de símbolos que una contraseña puede usar. Su tamaño, N, es la cantidad de opciones disponibles para cada posición, se arma a partir de cuatro subconjuntos: mayúsculas (26), minúsculas (26), dígitos (10) y símbolos (32). En este proyecto se comporta como un módulo de clasificación que revisa qué subconjuntos aparecen en la clave y de ahí obtiene N. El máximo es 94.

Principio aditivo

Es una regla de conteo. Si se tienen varios conjuntos que **no comparten elementos**, el total de opciones es la **suma** de lo que tiene cada uno. Se comporta como el «o» del conteo. Un carácter puede ser mayúscula **o** minúscula **o** dígito **o** símbolo, así que se suman las opciones: 26 + 26 + 10 + 32.

Principio multiplicativo

Siendo otra regla de conteo. Donde si un proceso tiene varias etapas **independientes**, el total de resultados es el **producto** de las opciones de cada etapa. En este caso es el «y» del conteo. Eliges el primer carácter **y** el segundo **y** el tercero, y así hasta completar la clave. Si cada uno tiene N opciones, se multiplican. Aquí cada posición de la contraseña es una etapa con N opciones, y las L posiciones se multiplican entre sí. Debido a esto, el espacio de claves crece **exponencialmente** con la longitud. Cada carácter nuevo lo multiplica por N, y es la razón matemática de que la longitud pese tanto.

Permutaciones con repetición y espacio muestral (W)

Una permutación es una selección **ordenada**. «Con repetición» significa que un mismo símbolo puede usarse varias veces, como en «aaa1». El espacio muestral W es el total de claves distintas que se pueden formar. Este es el resultado de aplicar el principio multiplicativo cuando todas las etapas tienen las mismas N opciones:

$$W = N^{L}.$$

W es el número que el motor combinatorio calcula primero, porque de él salen la entropía y el tiempo. Para $N = 94$ y $L = 8$, $W \approx 6{,}10 \times 10^{15}$. Por eso Python es adecuado para el programa, ya que maneja enteros enormes sin desbordarse.

Entropía (E)

Es una medida, en **bits**, de qué tan impredecible es una clave. Más bits significa más incertidumbre para el atacante. Este se obtiene con el logaritmo en base 2 de W:

$$E = \log_2(W) = L \cdot \log_2(N).$$

El logaritmo convierte un número gigantesco en una cifra fácil de comparar. Esta es la base de la clasificación en cinco niveles, de Muy Débil a Muy Fuerte. Su comportamiento crece **linealmente** con L.

Ataque de fuerza bruta y tasa de ataque (R)

La fuerza bruta es probar sistemáticamente todas las combinaciones hasta dar con la correcta. R es cuántos intentos por segundo puede hacer el atacante. Cuando no hay información previa, el atacante recorre el espacio de claves completo, o en promedio la mitad.

Tiempo de ruptura (T)

Es cuánto tardaría el atacante en recorrer todo el espacio de claves.

$$T = W / R$$

Es el peor caso, y el tiempo promedio es la mitad.

Es **inversamente proporcional** a R. Si el atacante es 1.000 veces más rápido, el tiempo baja 1.000 veces.

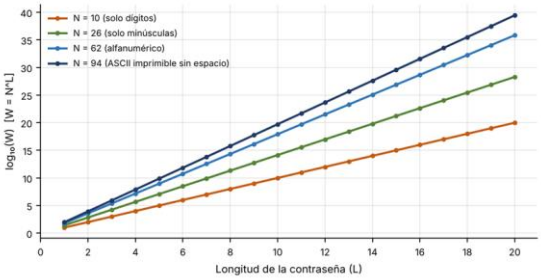


Figura 1. Crecimiento de log10(W) según la longitud L para distintos tamaños de alfabeto N

Marco Conceptual

Término	Definición
Alfabeto (N)	Conjunto de símbolos admitidos por una política de contraseñas; su tamaño N es la suma de los subconjuntos disjuntos (M, m, D, S).
Espacio muestral de claves (W)	Número total de combinaciones únicas que pueden formarse con las reglas seleccionadas; $W = N^L$.
Principio aditivo	Regla de conteo según la cual el cardinal de una unión de conjuntos finitos disjuntos es la suma de sus cardinales.
Principio multiplicativo	Regla de conteo según la cual el número de formas de completar etapas sucesivas e independientes es el producto de las formas de cada etapa.
Permutación con repetición	Selección ordenada de L elementos de un conjunto de N, en la que cada elemento puede repetirse; su número es N^L.
Ataque de fuerza bruta	Método sistemático que prueba todas las combinaciones posibles del espacio muestral hasta encontrar la correcta.
Entropía de la contraseña (E)	Medida matemática de la incertidumbre o impredecibilidad de una clave, expresada en bits: $E = \log_2(W)$.
Tasa de ataque (R)	Número de intentos por segundo que el atacante puede realizar; determina el tiempo de ruptura $T = W/R$.
Política de contraseñas	Conjunto de reglas (longitud mínima, tipos de caracteres, vigencia) que un sistema impone a las credenciales de sus usuarios.
Fortaleza	Nivel de resistencia de una política frente a la fuerza bruta, clasificado en cinco niveles: Muy Débil, Débil, Media, Fuerte y Muy Fuerte.

Tabla 1. Términos clave del proyecto

Diseño del sistema

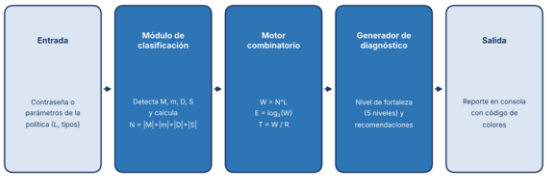


Figura 2. Arquitectura de bloques de PassShield

Conclusiones

Los resultados muestran que la longitud de la contraseña pesa más que la variedad de caracteres. Cada carácter nuevo multiplica el espacio de claves por N, mientras que ampliar el alfabeto de 62 a 94 símbolos en una clave de 8 caracteres solo lo multiplica por unas 28 veces. En cambio, agregar apenas cuatro caracteres con N = 62 lo multiplica por casi 14,8 millones. Este comportamiento es exponencial, lo que significa que la entropía crece de forma lineal con la longitud pero el tiempo de ruptura se dispara. También influye la capacidad del

atacante, porque el tiempo es inversamente proporcional a R. Por último, el modelo tiene un límite, ya que supone claves aleatorias y un atacante sin información previa. Una clave como «Pa55w0rd!» marca cerca de 59 bits, pero sigue un patrón muy conocido, así que su fortaleza real es mucho menor y los resultados deben leerse como una cota superior.

Los resultados indican que, para fortalecer una política, conviene priorizar la longitud sobre la complejidad de los caracteres, lo que confirma la hipótesis del proyecto. PassShield, una vez implementada y validada con los casos de prueba, permitirá comparar políticas de autenticación con datos y no con intuición. Sus limitaciones, como no detectar patrones de diccionario ni modelar funciones hash, marcan el camino para versiones futuras, entre ellas la detección de patrones comunes y una interfaz gráfica más accesible.

Referencias

[1] Python Software Foundation. (s. f.). The Python standard library (Versión 3.11) [Documentación]. https://docs.python.org/3.11/library/index.html

[2] Rosen, K. H. (2019). Matemática discreta y sus aplicaciones (8.ª ed.). McGraw-Hill Education.

[3] Shannon, C. E. (1948). A mathematical theory of communication. Bell System Technical Journal, 27(3), 379–423.

[4] Stallings, W., & Brown, L. (2018). Computer security: Principles and practice (4.ª ed.). Pearson

[5] Bonneau, J. (2012). The science of guessing: Analyzing an anonymized corpus of 70 million passwords. En 2012 IEEE Symposium on Security and Privacy (pp. 538–552). IEEE. https://doi.org/10.1109/SP.2012.49

[6] Cover, T. M., & Thomas, J. A. (2006). Elements of information theory (2.ª ed.). Wiley-Interscience.

[7] Temoshok, D., Choong, Y.-Y., Regenscheid, A., Galluzzo, R., Fenton, J. L., Richer, J., & Lefkovitz, N. (2025). Digital identity guidelines: Authentication and authenticator management (NIST Special Publication 800-63B-4). National Institute of Standards and Technology.